Lecture 17 · Part IV · Lecture notes

Text

Dataset: IMDb film reviews

How to use these notes

The primary text for this lecture is Géron, Chapter 14. These notes are not a replacement for it and do not repeat it: they are the lecture's own argument written out at length — the derivation done slowly, the numbers with the runs that produced them, and the reasoning between one slide and the next that a deck can only gesture at. Read the chapter for the method; read these for why the lecture makes the choices it does. Every figure quoted here is printed by this lecture's notebook, and the course checks that mechanically rather than trusting it.

Contents

1 A new kind of input

Sixteen lectures on numbers, pixels and ordered numbers. Today the input is free text: variable length, discrete, and with no natural numeric encoding at all. A review is a sequence of *symbols*, not of measurements, and there is no distance between two words that anything gave us.

Everything in Chapters 1–13 arrived as a table — a fixed set of features, in a fixed order. A review is none of those: variable length, no fixed positions, and an *open* vocabulary, because the next review can contain a word that has never been written before. So before any model there is a question no previous application had to answer: *what is the unit?*

1.1 The brief

A streaming service's feedback desk wants, for each new review, a single label — satisfied or not — so the unhappy ones can be routed to a human within the hour.

Their words	**What that specifies**
"tell us if it is a complaint"	binary classification
"from the text alone"	one variable-length string in, one label out
"within the hour"	*inference* cost matters; training cost does not
"we will read the ones you flag"	the errors are absorbed by a human

Four sentences, and every one is a design decision you would otherwise have made silently.

Counting the training vocabulary: 87,171 distinct words, of which 35,344 appear exactly once — 41%. Two words in five are seen once and never again. You cannot learn a vector for a word from one example, and you cannot afford a column for every one of them.

1.2 The anchor, before any model

Lecture 3's rule is to count the classes before choosing a metric. They are 50.0% to 50.0%, so accuracy is defensible *on this corpus* — the degenerate classifier scores 50% and has nowhere to hide. Real feedback is not balanced, and the first slide of the next application should ask again.

A bag of words weights each term by tf-idf = $\text{count}(t, d) \times \log \frac{N}{1+n_t}$, the second factor being what stops *the* and *movie* dominating the first. Note what is *not* in that formula: position. The bag has no order.

Model	Features	Test accuracy	Scoring 25,000
Always predict one class	—	50.0%	—
tf-idf, single words	44,798	88.5%	1.4 s
tf-idf, words and pairs	200,000	90.1%	3.6 s

90.1% from counting words, in ten seconds. Adding adjacent pairs buys a point and a half, and pairs are the cheapest way a bag can see *not good*. That last column is the brief's other requirement, which the metric cannot see, so it goes on the sheet too.

2 Derivation: softmax, cross-entropy and logits

Your model's last line returns two unbounded real numbers per review, with no softmax anywhere, and you hand them to `nn.CrossEntropyLoss` and it works. Why?

Logits are the output of the last linear layer before anything is done to it: unbounded, not summing to anything, and *only their differences carry information*. The name is historical — for $K = 2$ the difference $z_1 - z_0$ is the log-odds, the logit of Lecture 4, so logistic regression is this construction with two classes and one difference.

Softmax maps them to a probability vector, $\sigma(\mathbf{z})_k = e^{z_k} / \sum_j e^{z_j}$: every entry positive, summing to one, and *order-preserving* — which already tells you that `argmax` does not need it. A prediction never requires softmax.

2.1 Shift invariance, and why it is not a triviality

Adding a constant to every logit changes nothing: the two computations agree to 5.96×10^{-8}, which is float32 equality. The function ignores the shift.

float32 does not

e^x overflows float32 when $x > \log(3.4 \times 10^{38}) \approx 88.7$, and a confident network reaching logits of 100 is completely ordinary. The exponential is not representable, the sum becomes `inf`, the ratio becomes `nan` — and `nan` propagates into every parameter through `backward()`.

So the implementation subtracts the maximum:

$$\sigma(\mathbf{z})_k = \frac{e^{z_k - z_{\max}}}{\sum_j e^{z_j - z_{\max}}}.$$

The largest exponent is now exactly 0, the smallest may underflow to 0 which is harmless, and the denominator is at least 1 so the division is safe. *Shift invariance is not a curiosity about the formula. It is the licence to do this*, and without it there would be no numerically safe softmax.

2.2 The loss, and its gradient

Cross-entropy against a one-hot target collapses to $L = -\log p_c$: one number, and unbounded when the model is confidently wrong.

p_c	$-\log p_c$	**Reading**
1.00	0.00	certain and right
0.50	0.69	a coin, on two classes
0.10	2.30	wrong, and not yet confident about it
0.01	4.61	wrong and certain — unbounded penalty

$-\log$ is what makes confident mistakes expensive; a loss linear in p_c would price the last two rows almost the same. And on two balanced classes an untrained model should show a loss near 0.69 — if your first epoch starts far from that, something is wrong before any training has happened.

The gradient with respect to the logits

Substituting softmax into the loss,

$$L = -\log \frac{e^{z_c}}{\sum_j e^{z_j}} = -z_c + \log \sum_j e^{z_j}.$$

The exponential of the true class has cancelled. Differentiating, the first term contributes $-y_k$ and the second is the log-sum-exp, whose derivative is softmax:

$$\boxed{\frac{\partial L}{\partial \mathbf{z}} = \mathbf{p} - \mathbf{y}}$$

Prediction minus target. Three lines, no chain rule through the softmax and no Jacobian. Every component lies in $[-1, 1]$, so the signal into the network is bounded however wrong the model is; the components sum to zero, because both vectors sum to one — so the gradient has no component along $\mathbf{1}$, which is exactly the direction softmax could not see; and it vanishes only when the prediction is exactly the target.

Compare mean squared error on probabilities, whose gradient carries a factor $p_k(1 - p_k)$ — near zero precisely when the model is confidently wrong, which is when you most need a gradient. Checked against autograd, the algebra and PyTorch agree to 5.55×10^{-17}.

2.3 So why does the loss want logits?

Because of that cancellation. Given probabilities you must form every exponential, divide, and then take the logarithm of a number that may be 10^{-40}. Given logits it is one pass with the maximum subtracted inside the log-sum-exp, and *the exponential and the logarithm never meet.*

The failure that does not announce itself

Take logits $[0, 0, -100]$ in float32 with the true class last.

Computed as	Loss	Error
float64 reference	100.69315	—
naive, probabilities first	100.63987	0.0533
combined, from logits	100.69315	1.43×10^{-6}

Nothing overflowed. The probability $e^{-100}/(2 + e^{-100})$ is 1.86×10^{-44}, which float32 can hold only as a subnormal with a handful of significant bits: the naive path lands on 1.96×10^{-44} instead, and the logarithm of that is wrong in the second decimal — finitely, plausibly, and without a warning.

Finally, cross-entropy and KL divergence differ by the entropy of the target, $H(\mathbf{y}, \mathbf{p}) = H(\mathbf{y}) + D_{\mathrm{KL}}(\mathbf{y} \| \mathbf{p})$ — add and subtract $\sum_k y_k \log y_k$, and that is the whole proof. For a one-hot target $H(\mathbf{y}) = 0$, so minimising cross-entropy *is* minimising the KL divergence to the label.

3 Tokens, embeddings, and a classifier

3.1 What is a token?

The unit the model counts as one thing. *Characters* give a tiny vocabulary, sequences six times longer, and every fact about a word relearned from letters. *Words* are the obvious choice, and the one that breaks. *Subwords* are the compromise the field settled on.

This is a modelling decision, not a preprocessing detail: it fixes the vocabulary, the sequence length, and what the model can possibly represent.

Our word tokenizer is one regular expression, and three arguable decisions are already inside it: case is destroyed, so *Terrible* and *terrible* become one token; punctuation is destroyed, and so is the exclamation mark; and the corpus is HTML, so `<br />` has to be removed. *If you had not looked at the raw text you would have learned* `br` *as one of the commonest words in the corpus.*

What a word vocabulary costs

Even at the full 79,003-word vocabulary of the fit split — smaller than the 87,171 distinct words above, because a model may not see the validation reviews — **42.8%** of the *distinct* words in the test set have never been seen.

The token rate looks survivable. The rate over distinct words never does, and those are the informative ones.

Subword tokenisation is a structural fix rather than a patch. Start from characters, which cover everything by construction, and repeatedly merge the commonest adjacent pair until the vocabulary reaches the size you asked for. Common words survive whole; rare words become pieces. *The vocabulary cannot miss*: an unseen word falls back to pieces, an unseen piece to characters, and the character set is closed — so there is no input for which the tokenizer must emit `[UNK]`. Morphology survives too: *un* + *watch* + *able* keeps the negation that `[UNK]` destroyed.

3.2 From integers to vectors

Token 4271 is not four thousand of anything. The integer is a *name*, and arithmetic on names is meaningless. One-hot encoding would give a 20,000-dimensional vector per token and 256 of them per review.

An **embedding** is a learned table with one row per token: looking up row i is exactly multiplying the one-hot vector by that table, done without ever forming the one-hot vector. It is a *parameter*, not a preprocessing step — every one of its numbers has a gradient — and `padding_idx=0` pins row 0 at zero and keeps it there, because padding must not learn anything.

The classifier is thirteen lines, eleven of which are Lecture 10. Bidirectional because a review's verdict can be in the first sentence or the last; the final hidden state because it is one fixed-size summary of a variable-length input; GRU rather than LSTM because Lecture 16 measured no difference here.

`pack_padded_sequence`, and what it prevents

It hands the recurrent layer only the real timesteps, so the state returned for a review of 40 words is the state after its 40th word — not after 216 zeros.
Correctness, because the summary is of the review rather than of the padding; and speed, because the layer stops early instead of stepping through the whole rectangle.
This matters because the version *without* it also runs, also trains, and also reports a plausible number.

Setting	Value
Reviews used to fit	20,000
Reviews held out for validation	5,000
Reviews in the test set, untouched	25,000
Parameters	2,634,754
Wall clock, one run	460 s

Model	Test accuracy
Always one class	50.0%
tf-idf + logistic regression	90.1%
GRU, our tokenizer, trainable embeddings	83.4%

The recurrent network, with an embedding it learned itself, loses to counting words.

Why? The bag starts with 200,000 features that are already meaningful — *terrible* is its own column. Our network starts with 20,000 rows of random numbers and has to discover from 20,000 labelled reviews that *terrible* and *awful* are related, when two words in five occur once.

The hypothesis — and it is only a hypothesis

The architecture is not the bottleneck. The embedding table is, and it is being asked to learn the English language from 20,000 sentiment labels.
Written as a hypothesis on purpose. What follows is an experiment that can refute it, and you should decide now what result would.

4 The swap

Replace the table, and nothing else: same GRU, same width, same head, same optimiser, same learning rate, same four epochs, same seed, same 20,000 reviews. Two changes, applied *one at a time*, because changing both and reporting the total is how you learn nothing from an experiment.

Tokenizer	Embedding table starts as	Test accuracy
our words	random	83.4%
subword	random	82.5%
subword	pretrained, frozen	82.6%
subword	pretrained, tuned	84.2%

The subword vocabulary alone is −0.94 points, from one seed each — not enough to call a ranking, and we will not. What the measurement *does* say is that solving the out-of-vocabulary problem, on its own, bought nothing.

The pretrained vectors, fine-tuned, are worth 0.79 points, and not one line of the model changed. Positive, and small — about the size of the swing we refused to call a ranking one row above. Everything ordinarily tuned in an afternoon (width, depth, head, loss, optimiser, epochs, batch size, learning rate, seed) is unchanged; what moved is the tokenizer, the initial values in one tensor, and whether that tensor is trainable.

A warning about that validation set

From the corpus README, not from us: "the train and test sets contain a disjoint set of movies, so no significant performance is obtained by memorizing movie-unique terms". Our validation split was carved out of the training half, so it *shares films* with the fit split — and a model that memorises film-specific vocabulary is flattered by it. The test half shares no films with anything, which is why the drop from validation to test is larger for the run that overfits most.

A validation set drawn from the same pool as the training data answers "when should I stop?" It does not answer "how well will this do?"

Model	Test accuracy	To fit	To score 25,000
Always one class	50.0%	—	—
tf-idf + logistic regression	90.1%	10 s	3.6 s
GRU from scratch	83.4%	460 s	6.3 s
GRU, pretrained embeddings	84.2%	302 s	12.0 s

Two clocks, and they are not the same clock. The one that matters to the desk is the last column, because it is paid once per review forever; the middle one is paid once. *The best accuracy here is still 5.9 points behind counting words, at more than three times the cost to run.*

Text checklist

1. Was the vocabulary built on the training split only?
2. Is the tokenizer the same object at fit time and at inference?
3. Are true lengths carried alongside the padded batch?
4. Does padding change the output? Assert that it does not.
5. What fraction of the test tokens is `[UNK]`? Count it.
6. How much of each review survives truncation? Count that too.

The first is the one that will cost you the most, and it is the one the next lecture ends on.

5 The notebook

What to change

Apply a softmax before a loss that expects logits. Nothing raises, the model still trains, and it trains *worse* — the double-softmax failure, measured rather than warned about.

6 Exercises

1. Show that softmax is invariant to adding a constant to every logit, and state the numerical reason the implementation subtracts the maximum anyway. [5]
2. Derive $\partial L/\partial \mathbf{z} = \mathbf{p} - \mathbf{y}$ for cross-entropy on a one-hot target, and state what the row sum of that gradient must be. [5]
3. A loss function is given probabilities rather than logits. State the two failure modes, and which one ships. [5]
4. A model summarises a padded batch at `out[:, -1, :]`. State what that position holds for most reviews, and the invariance test that detects it. [5]
5. Enlarging a word vocabulary drives the share of unseen distinct test words down, but even the full 79,003-word vocabulary leaves 42.8% of them unseen. Explain why a larger vocabulary is not the fix, and name what is. [4]

Summary

Softmax is invariant under adding a constant to every logit, which is the licence to subtract the maximum before exponentiating — necessary because e^x overflows float32 above about 88.7 and a confident network reaching logits of 100 is ordinary. Cross-entropy against a one-hot target is $-\log p_c$, unbounded when the model is confidently wrong, and its gradient with respect to the logits is exactly $\mathbf{p} - \mathbf{y}$: bounded, summing to zero, free of the softmax Jacobian, and agreeing with autograd to 5.55×10^{-17}.

The loss consumes logits because $-z_c$ cancels the exponential. Computing it the other way does not always overflow — on logits $[0, 0, -100]$ it returns 100.63987 against a true 100.69315, wrong in the second decimal, finitely and plausibly and without a warning.

Then the application, which did not go the way the phrase “transfer learning” suggests. A word vocabulary leaves 42.8% of distinct test words unseen even at 79,003 entries, and subword tokenisation removes that failure entirely — and bought *nothing* on its own. Pretrained vectors, fine-tuned, were worth 0.79 points. And every version of the recurrent model lost to tf-idf and logistic regression, which scores 90.1% in ten seconds.